



## PROJECT AND TEAM INFORMATION

### Project Title

**OS User Activity Monitoring & Insider Threat Detection System**

|                                    |                                                                                                                                                                 |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Team Name:</b><br><b>Team #</b> | <b>KernelGuard</b><br><b>OSDBMS-V-2026-T298</b><br><b>Mentor: (Prof.)Dr.Mohammad Wazid</b>                                                                      |
| <b>Team member 1 (Team Lead)</b>   | <b>Arshpreet Singh</b><br><b>24021835</b><br><b>24021835@geu.ac.in</b><br> |
| <b>Team member 2</b>               | <b>Mohd Ahmed Khan</b><br><b>24021365</b><br><b>24021365@geu.ac.in</b><br> |
| <b>Team member 3</b>               | <b>Ankit</b><br><b>240212134</b><br><b>240212134@geu.ac.in</b><br>         |
| <b>Team member 4</b>               | <b>Mohd Shoaib</b><br><b>240211124</b><br><b>240211124@geu.ac.in</b><br>   |

## PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

### Project Abstract (2 pts)

KernelGuard Phase II demonstrates a traceable path from Linux Audit output to an administrator alert. A collector groups related audit records, extracts selected identity and file fields, and stores them in a dedicated MySQL database. Two rules identify repeated authentication failures and successful protected-file access outside configured hours. The dashboard shows each alert with its supporting events and policy values. The nine-event fixture is clearly labelled synthetic, while live records carry a separate origin.

### Updated Project Approach and Architecture (2 pts)

A Linux audit rule watches a protected directory. The parser groups lines by audit event identity and normalizes timestamps, outcome, UID fields and path metadata. A unique source identifier prevents duplicate imports. The collector obtains a MySQL writer lock and stores events, alerts and its checkpoint in one transaction. SQLAlchemy defines eight tables with keys, foreign keys and indexes. Flask serves a localhost dashboard through a SELECT-only database account. A separate fixture imports controlled events for repeatable classroom evaluation. This phase deliberately limits detection to two rules so the evidence path can be inspected end to end.

## Tasks Completed (7 pts)

| Task Completed                               | Team Member     |
|----------------------------------------------|-----------------|
| Audit rule, parser and checkpoint collector  | Arshpreet Singh |
| Eight-table MySQL schema and SQL queries     | Mohd Ahmed Khan |
| Two policies and fixture checks              | Ankit           |
| Read-only dashboard and alert evidence pages | Mohd Shoaib     |

## Challenges/Roadblocks (7 pts)

Linux Audit emits several lines for one action, so the parser must form a complete event before applying a rule. Collector restarts can replay records; unique source IDs and a checkpoint in the write transaction address that case. The fixture makes demonstrations reproducible but cannot establish live collection. A controlled protected-file read and MySQL 8.4 integration check were therefore recorded separately on the Kali lab host.

## Tasks Pending (7 pts)

| Task Pending                                        | Team Member (to complete the task) |
|-----------------------------------------------------|------------------------------------|
| User, session, process and USB inventory            | Phase III                          |
| Three additional policy families and review history | Phase III                          |
| Physical USB acceptance check                       | Phase III                          |

## Project Outcome/Deliverables (2 pts)

The Phase II deliverable is a working two-rule Linux Audit and MySQL prototype. It includes the collection rule, parser, eight-table relational model, fixture, SQL analysis queries, read-only dashboard and tests. The fixture produces nine events, two alerts and six evidence links. A Kali check dated 2 October 2026 also recorded a live protected-file read and a passing Oracle MySQL 8.4 integration run.

## Progress Overview (2 pts)

The audit-to-alert path and local dashboard are implemented. The saved Kali check covers a controlled live file read and the MySQL-backed workflow. Broader host inventory, privileged-command and bulk-file rules, USB enrollment, and administrator decisions belong to the Phase III scope. Phase II results should be presented as a focused prototype; fixture counts are synthetic and do not measure field detection accuracy.

Codebase Information (2 pts)

Local source: C:\KernalGuard-Phase2-PBL. Branch: main. This standalone copy has no commit history or configured public remote. Main files: app.py, kernelguard/parser.py, kernelguard/core.py, kernelguard/locking.py, linux/kernelguard.rules, sql/schema.mysql.sql and tests/test\_app.py. Technical basis: Red Hat audit-log documentation, SQLAlchemy transaction documentation and the MySQL 8.4 foreign-key manual.

Testing and Validation Status (2 pts)

| Test Type                 | Status (Pass/Fail)  | Notes                           |
|---------------------------|---------------------|---------------------------------|
| Local automated tests     | 3 passed, 1 skipped | MySQL test is opt-in            |
| MySQL 8.4 integration     | 4 passed            | Recorded Kali run, 2 Oct 2026   |
| Controlled protected read | Pass                | Live audit row recorded on Kali |

Deliverables Progress (2 pts)

Completed: collector, parser, two policies, relational schema, SQL queries, fixture, dashboard and setup guide. Demonstrated separately: live Kali read and MySQL 8.4 integration. The standalone Phase II copy documents this milestone; it is not a historical Git snapshot of the original submission.